

Containere software și orchestrarea acestora

Autor: Alexandra Moldovan

Data: Iunie 2026

0 Cuprins

1	Introducere	3
2	Virtualizare și containere software	4
2.1	Conceptul de virtualizare	4
2.2	Limitările mașinilor virtuale	4
2.3	Conceptul de container software	4
2.4	Mecanismele de funcționare ale containerelor	5
3	Docker — Platforma standard pentru containere	5
3.1	Prezentare generală	5
3.2	Arhitectura Docker	6
3.2.1	Docker Client	6
3.2.2	Docker Daemon	6
3.2.3	Docker Registry	7
3.3	Imaginile Docker	7
3.3.1	Definiție și structură	7
3.3.2	Nomenclatură și versionare	7
3.4	Dockerfile	8
3.4.1	Rol și structură	8
3.4.2	Instrucțiuni principale	8
3.4.3	Exemplu de Dockerfile	8
3.5	Containerele Docker	9
3.6	Rețele și volume Docker	9
3.6.1	Rețele Docker	9
3.6.2	Volume Docker	9
3.7	Docker Compose	10
4	Orchestrarea containerelor	10
4.1	Necesitatea orchestrării	10
4.2	Funcționalitățile principale ale orchestrării	11
4.3	Platforme de orchestrare	11
5	Kubernetes — Arhitectura și componentele principale	11
5.1	Prezentare generală	12
5.2	Arhitectura generală a unui cluster Kubernetes	12
5.3	Componentele Control Plane	12
5.3.1	API Server (kube-apiserver)	12

5.3.2	etcd	12
5.3.3	Scheduler (kube-scheduler)	13
5.3.4	Controller Manager (kube-controller-manager)	13
5.3.5	Cloud Controller Manager	13
5.4	Componentele Worker Node	13
5.4.1	kubelet	13
5.4.2	kube-proxy	14
5.4.3	Container Runtime	14
5.5	Obiectele fundamentale Kubernetes	14
5.5.1	Pod	14
5.5.2	ReplicaSet și Deployment	14
5.5.3	Service	15
5.5.4	ConfigMap și Secret	15
5.5.5	Namespace	15
5.5.6	Ingress	15
5.5.7	Persistent Volumes	16
6	Studiu de caz: Containerizarea unei aplicații bazate pe microservicii	16
6.1	Prezentarea aplicației	16
6.2	Containerizarea cu Docker	16
6.3	Implementarea în Kubernetes	16
6.4	Scalare automată și toleranță la erori	17
7	Avantaje, provocări și tendințe	17
7.1	Avantajele utilizării containerelor și orchestrării	17
7.2	Provocări și limitări	17
7.3	Tendințe și direcții viitoare	18
8	Concluzii	18

1 Introducere

Transformarea digitală accelerată a ultimului deceniu a determinat modificări fundamentale în modul în care aplicațiile software sunt concepute, dezvoltate și distribuite. Adoptarea pe scară largă a arhitecturilor distribuite și a paradigmei cloud computing a generat necesitatea unor soluții tehnice capabile să răspundă cerințelor de scalabilitate, portabilitate și disponibilitate ale aplicațiilor moderne.

În acest context, metodele tradiționale de distribuire a aplicațiilor, bazate pe instalarea directă pe sisteme de operare sau pe utilizarea mașinilor virtuale, au început să manifeste limitări semnificative. Consumul ridicat de resurse, timpii mari de inițializare și dificultățile legate de portabilitate au constituit obstacole majore în calea automatizării și a livrării rapide de software.

Containerele software au apărut ca răspuns la aceste provocări, oferind un mecanism standardizat de împachetare a aplicațiilor împreună cu toate dependențele necesare rulării acestora. Prin intermediul containerelor, aplicațiile pot fi executate într-un mod consistent, indiferent de mediul de execuție, reducând astfel problemele de compatibilitate și simplificând procesul de livrare continuă a software-ului.

Odată cu creșterea numărului de containere utilizate în mediile de producție, administrarea manuală a acestora a devenit dificilă și ineficientă. Pentru a răspunde acestei probleme au fost dezvoltate platformele de orchestrare a containerelor, care permit automatizarea proceselor de implementare, monitorizare, scalare și recuperare în caz de eroare. Kubernetes s-a impus ca standard de facto în industrie, fiind adoptat de o mare parte dintre organizațiile care operează infrastructuri cloud.

Prezenta lucrare analizează conceptele fundamentale asociate containerelor software și orchestrării acestora, cu accent pe arhitectura și componentele platformelor Docker și Kubernetes. Obiectivele principale sunt:

- prezentarea conceptului de container software și a mecanismelor de funcționare la nivel de sistem de operare;
- analiza detaliată a arhitecturii Docker și a componentelor sale principale;
- studierea mecanismelor de orchestrare a containerelor și a necesității acestora;
- prezentarea arhitecturii Kubernetes și a obiectelor sale fundamentale;
- ilustrarea aplicării practice a acestor tehnologii printr-un studiu de caz.

2 Virtualizare și containere software

2.1 Conceptul de virtualizare

Virtualizarea reprezintă o tehnologie care permite rularea mai multor sisteme de operare și aplicații pe aceeași infrastructură hardware fizică, prin intermediul unui strat software de abstractizare. Această tehnologie a devenit una dintre inovațiile fundamentale ale centrelor de date moderne, deoarece permite utilizarea mai eficientă a resurselor hardware și simplifică administrarea infrastructurii informatice.

În cadrul virtualizării tradiționale, un hipervizor (*hypervisor*) este responsabil pentru gestionarea mașinilor virtuale și pentru distribuirea resurselor hardware disponibile între acestea. Fiecare mașină virtuală include propriul sistem de operare complet, propriile biblioteci și aplicațiile necesare funcționării.

Există două categorii principale de hipervizoare:

- **Hipervizoare de tip 1 (bare-metal)** — rulează direct pe hardware-ul fizic și oferă performanțe ridicate. Exemple reprezentative: VMware ESXi, Microsoft Hyper-V și Xen.
- **Hipervizoare de tip 2 (hosted)** — rulează peste un sistem de operare existent și sunt utilizate frecvent în scopuri de dezvoltare și testare. Exemple: Oracle VirtualBox și VMware Workstation.

2.2 Limitările mașinilor virtuale

Deși mașinile virtuale oferă un nivel ridicat de izolare și flexibilitate, utilizarea lor prezintă dezavantaje semnificative în contextul aplicațiilor moderne:

- consum ridicat de memorie și spațiu de stocare, datorat sistemului de operare complet inclus în fiecare instanță;
- timp de inițializare de ordinul minutelor;
- dificultăți în distribuirea rapidă și reproductibilă a aplicațiilor;
- complexitate crescută în administrarea unui număr mare de instanțe;
- costuri suplimentare asociate licențierii sistemelor de operare.

Odată cu adoptarea arhitecturilor bazate pe microservicii, aceste limitări au determinat apariția unor soluții mai eficiente.

2.3 Conceptul de container software

Containerele software reprezintă o formă de virtualizare la nivelul sistemului de operare. Spre deosebire de mașinile virtuale, containerele nu includ un sistem de

operare complet, ci utilizează nucleul (*kernel-ul*) sistemului gazdă pentru a executa aplicațiile. Un container conține toate componentele necesare funcționării aplicației: codul sursă, bibliotecile, fișierele de configurare și dependențele software.

Comparație: mașini virtuale vs. containere

Criteriu	Mașini virtuale	Containere
Pornire	~minute	~secunde
Memorie	GB / instanță	MB / instanță
OS propriu	Da (complet)	Nu (shared kernel)
Izolare	Completă	Parțială (namespace)
Portabilitate	Limitată	Excelentă

2.4 Mecanismele de funcționare ale containerelor

Funcționarea containerelor este posibilă datorită unor mecanisme oferite de kernel-ul Linux:

- **Namespaces** — asigură izolarea proceselor, a rețelei, a sistemului de fișiere și a utilizatorilor. Fiecare container dispune de propriul spațiu de nume, astfel încât procesele sale nu pot interacționa cu cele ale altor containere sau ale sistemului gazdă.
- **Control Groups (cgroups)** — permit limitarea și monitorizarea consumului de resurse hardware (procesor, memorie, bandă de rețea, acces la disc) pentru fiecare container în parte.
- **Union File Systems (UnionFS)** — permit construirea imaginilor containerelor din mai multe straturi (*layers*) reutilizabile, suprapuse în mod transparent. Această abordare optimizează spațiul de stocare și accelerează distribuirea imaginilor.

Prin combinarea acestor mecanisme, containerele oferă izolare adecvată și eficiență ridicată, fără costurile asociate rulării unui sistem de operare complet.

3 Docker — Platforma standard pentru containere

3.1 Prezentare generală

Docker este o platformă open-source pentru dezvoltarea, distribuirea și rularea aplicațiilor în containere, lansată în anul 2013 de compania dotCloud (ulterior redenumită Docker Inc.). Platforma a contribuit decisiv la popularizarea tehnologiei containerelor,

oferind un ecosistem complet și un model de utilizare standardizat, accesibil atât dezvoltatorilor, cât și administratorilor de infrastructură.

Scopul fundamental al Docker este eliminarea discrepanțelor dintre mediile de dezvoltare, testare și producție, prin intermediul principiului „*build once, run anywhere*”. O aplicație împachetată într-un container Docker se execută în mod identic pe orice sistem pe care este instalat motorul Docker, indiferent de sistemul de operare sau configurația hardware a mașinii gazdă.

3.2 Arhitectura Docker

Docker utilizează o arhitectură de tip client-server, compusă din mai multe componente care interacționează prin intermediul unui API REST.

Componentele principale ale arhitecturii Docker

- **Docker Client** — interfața prin care utilizatorul emite comenzi
- **Docker Daemon (dockerd)** — serviciul de fundal care execută operațiile
- **Docker Images** — șabloanele imutabile pentru crearea containerelor
- **Docker Containers** — instanțele active ale imaginilor
- **Docker Registry** — depozitul centralizat de imagini

3.2.1 Docker Client

Docker Client (`docker`) reprezintă interfața principală de interacțiune cu platforma. Acesta acceptă comenzi din linia de comandă și le transmite, prin intermediul API-ului REST, către Docker Daemon, care realizează efectiv operațiile solicitate. Docker Client poate comunica cu un daemon local sau cu unul de pe un sistem remote.

Principalele categorii de comenzi disponibile sunt:

- comenzi pentru gestionarea imaginilor: `docker build`, `docker pull`, `docker push`, `docker images`, `docker rmi`;
- comenzi pentru gestionarea containerelor: `docker run`, `docker start`, `docker stop`, `docker rm`, `docker ps`;
- comenzi pentru inspecție și depanare: `docker logs`, `docker exec`, `docker inspect`, `docker stats`.

3.2.2 Docker Daemon

Docker Daemon (`dockerd`) este procesul de fundal responsabil pentru gestiunea tuturor resurselor Docker: imagini, containere, volume și rețele. Acesta ascultă cererile primite prin API și le procesează conform instrucțiunilor transmise de client. Daemonul poate comunica cu alți daemoni Docker pentru a administra servicii dis-

tribuite.

3.2.3 Docker Registry

Docker Registry reprezintă un depozit centralizat pentru stocarea și distribuirea imaginilor Docker. Cel mai cunoscut registru public este **Docker Hub**, care găzduiește milioane de imagini, atât oficiale (menținute de Docker și de furnizorii de software), cât și contribuie de comunitate.

Principalele operații asupra registrelor sunt:

- `docker pull <image>` — descarcă o imagine din registry în cache-ul local;
- `docker push <image>` — încarcă o imagine locală în registry;
- `docker search <termen>` — caută imagini disponibile în registry-ul public.

Pe lângă Docker Hub, organizațiile pot opera registre private (de exemplu, Harbor, AWS ECR, Azure Container Registry, Google Artifact Registry) pentru distribuirea imaginilor interne, fără expunere publică.

3.3 Imaginile Docker

3.3.1 Definiție și structură

O imagine Docker este un șablon read-only, stratificat, utilizat pentru crearea containerelor. Aceasta conține toate elementele necesare executării aplicației: sistemul de bază, bibliotecile software, dependențele, fișierele de configurare și codul sursă.

Imaginile Docker sunt construite pe baza unui sistem de straturi suprapuse (*overlay layers*). Fiecare instrucțiune dintr-un *Dockerfile* generează un nou strat, iar straturile comune mai multor imagini sunt stocate o singură dată și reutilizate, ceea ce optimizează semnificativ spațiul de stocare și timpul de transfer al imaginilor.

O imagine este **imuabilă**: odată construită, nu poate fi modificată. Orice schimbare necesită construirea unei imagini noi. La rularea unui container, Docker adaugă deasupra straturilor read-only ale imaginii un strat de scriere (*container layer*), în care sunt stocate modificările efectuate pe durata execuției containerului.

3.3.2 Nomenclatură și versionare

Imaginile Docker sunt identificate printr-un nume și un tag, conform convenției:

```
[registry/] [utilizator/]nume[:tag]
```

Tagul implicit este `latest`, dacă nu este specificat altul. Utilizarea de taguri explicite (de exemplu, `python:3.11-slim`, `nginx:1.25-alpine`) este considerată bună practică, deoarece asigură reproductibilitatea build-urilor.

3.4 Dockerfile

3.4.1 Rol și structură

Dockerfile este un fișier text care conține un set de instrucțiuni pentru construirea automată a unei imagini Docker. Acesta definește imaginea de bază, pașii de configurare și comanda implicită de pornire a containerului. Utilizarea Dockerfile asigură reproductibilitatea mediului de execuție și integrarea facilă în pipeline-urile CI/CD.

3.4.2 Instrucțiuni principale

Principalele instrucțiuni disponibile în Dockerfile sunt:

- FROM — specifică imaginea de bază de la care pornește construcția;
- WORKDIR — setează directorul de lucru implicit în interiorul containerului;
- COPY / ADD — copiază fișiere din contextul de build în imagine;
- RUN — execută comenzi în timpul construirii imaginii (instalare pachete, compilare etc.);
- ENV — definește variabile de mediu disponibile în container;
- EXPOSE — documentează porturile pe care containerul le ascultă;
- CMD / ENTRYPOINT — specifică comanda executată la pornirea containerului.

3.4.3 Exemplu de Dockerfile

Următorul exemplu prezintă un Dockerfile pentru o aplicație web dezvoltată în Python, utilizând framework-ul Flask:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV FLASK_ENV=production
EXPOSE 5000

CMD ["python", "app.py"]
```

Listing 1: Exemplu de Dockerfile pentru o aplicație Python/Flask

Utilizarea imaginii `python:3.11-slim` în locul variantei `python:3.11` reduce dimensiunea imaginii finale cu aproximativ 60%, prin excluderea pachetelor de sistem neutilizate.

3.5 Containerele Docker

Un container Docker reprezintă o instanță activă a unei imagini. Dacă imaginea poate fi asimilată unui șablon static, containerul reprezintă procesul aflat în execuție, izolat de restul sistemului prin mecanismele de *namespace* și *cgroup* ale kernel-ului Linux.

Caracteristici esențiale ale containerelor:

- **Izolare** — fiecare container dispune de propriul sistem de fișiere, rețea și spațiu de procese;
- **Eficiență** — mai multe containere partajează același kernel al sistemului gazdă, fără overhead-ul unui OS complet;
- **Pornire rapidă** — un container pornește în secunde sau fracțiuni de secundă;
- **Efemeralitate** — în mod implicit, modificările efectuate în interiorul unui container nu sunt persistate după oprirea acestuia; pentru persistența datelor se utilizează *volume* Docker.

3.6 Rețele și volume Docker

3.6.1 Rețele Docker

Docker oferă mai multe tipuri de rețele pentru comunicarea între containere și cu exteriorul:

- **bridge** (implicit) — rețea privată virtuală pe mașina gazdă; containerele din aceeași rețea bridge pot comunica între ele prin nume;
- **host** — containerul utilizează direct stiva de rețea a gazdei, fără izolare;
- **none** — containerul nu are acces la rețea;
- **overlay** — utilizată în cluster-uri Docker Swarm pentru comunicarea între containere de pe mașini diferite.

3.6.2 Volume Docker

Volumele Docker reprezintă mecanismul recomandat pentru persistența datelor generate sau utilizate de containere. Spre deosebire de stratul de scriere al containerului (care se pierde la ștergerea acestuia), volumele există independent de ciclul de viață al containerelor și pot fi partajate între mai multe containere simultan.

3.7 Docker Compose

Docker Compose este un instrument pentru definirea și rularea aplicațiilor multi-container. Prin intermediul unui fișier `docker-compose.yml`, se pot specifica toate serviciile, rețelele și volumele necesare unei aplicații, putând fi lansate simultan cu o singură comandă (`docker compose up`).

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
    environment:
      DATABASE_URL: postgresql://user:pass@db/mydb

  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Listing 2: Exemplu de fișier `docker-compose.yml`

4 Orchestrarea containerelor

4.1 Necesitatea orchestrării

Containerele rezolvă problema portabilității și consistenței mediului de execuție la nivel individual. Însă în mediile de producție moderne, o aplicație bazată pe microservicii poate fi compusă din zeci sau sute de containere distribuite pe multiple servere. Administrarea manuală a acestora ridică probleme dificil de soluționat fără automatizare:

- repornirea automată a containerelor oprite din cauza erorilor;
- distribuirea echilibrată a containerelor pe nodurile disponibile;
- scalarea orizontală în funcție de încărcarea curentă a sistemului;

- actualizarea aplicațiilor fără întreruperea serviciului;
- redistribuirea sarcinilor în caz de defecțiune hardware.

4.2 Funcționalitățile principale ale orchestrării

Platformele moderne de orchestrare oferă un set cuprinzător de funcționalități:

- **Implementare declarativă** — starea dorită a sistemului este declarată, iar platforma asigură menținerea acesteia;
- **Scalare automată** — creșterea sau reducerea numărului de instanțe în funcție de metrice de utilizare;
- **Echilibrarea traficului** — distribuirea cererilor între instanțele disponibile ale aceluiași serviciu;
- **Self-healing** — detectarea și repornirea automată a containerelor și a nodurilor defecte;
- **Rolling updates și rollback** — actualizări progresive fără downtime, cu posibilitate de revenire rapidă;
- **Descoperirea serviciilor** — mecanisme automate prin care serviciile se localizează reciproc în interiorul clusterului.

4.3 Platforme de orchestrare

Principalele platforme de orchestrare a containerelor disponibile în prezent sunt:

- **Kubernetes** — standardul dominant în industrie, susținut de Cloud Native Computing Foundation (CNCF);
- **Docker Swarm** — soluție nativă Docker, mai simplă dar cu funcționalități limitate față de Kubernetes;
- **Apache Mesos** — platformă de orchestrare generală pentru resurse distribuite;
- **HashiCorp Nomad** — orchestrator flexibil, compatibil cu sarcini de lucru containerizate și necontainerizate.

Kubernetes a devenit standard de facto datorită flexibilității, ecosistemului extins și suportului nativ oferit de principalii furnizori de servicii cloud (Amazon EKS, Google GKE, Azure AKS).

5 Kubernetes — Arhitectura și componentele principale

5.1 Prezentare generală

Kubernetes (denumit frecvent K8s) este o platformă open-source de orchestrare a containerelor, proiectată inițial de Google pe baza experienței acumulate cu sistemul intern Borg, și donată comunității în 2014 prin intermediul Cloud Native Computing Foundation (CNCF). Platforma automatizează implementarea, scalarea și administrarea aplicațiilor containerizate în medii distribuite.

Principii fundamentale Kubernetes

- **Declarativ** — se declară starea dorită, nu pașii de atingere a acesteia
- **Autorecuperare** — starea reală converge automat spre starea declarată
- **Extensibil** — API-ul poate fi extins cu tipuri de resurse personalizate (CRD)
- **Portabil** — rulează on-premise, în cloud public sau hibrid

5.2 Arhitectura generală a unui cluster Kubernetes

Un cluster Kubernetes este format din două categorii de noduri:

- **Control Plane** — ansamblul componentelor responsabile cu administrarea globală a clusterului și cu luarea deciziilor de orchestrare;
- **Worker Nodes** — serverele pe care sunt programate și executate efectiv containerele aplicațiilor utilizatorilor.

5.3 Componentele Control Plane

5.3.1 API Server (kube-apiserver)

API Server reprezintă componenta centrală a Control Plane și singurul punct de intrare pentru toate operațiile asupra clusterului. Acesta expune API-ul RESTful al Kubernetes, prin care sunt procesate toate cererile venite de la clienți (`kubectl`, alte componente interne sau aplicații externe). API Server validează și autentifică cererile, le aplică politicile de autorizare (RBAC) și le persistă în *etcd*.

5.3.2 etcd

etcd este un sistem de stocare distribuit de tip cheie-valoare, consistent și tolerant la partiții, utilizat ca sursă unică de adevăr (*source of truth*) pentru starea întregului cluster Kubernetes. Toate configurațiile, starea curentă și metadatele obiectelor Kubernetes sunt stocate în etcd. Disponibilitatea și integritatea datelor din etcd sunt critice pentru funcționarea clusterului; din acest motiv, în mediile de producție, etcd este operat în configurație de cluster cu număr impar de membri (3 sau 5), pentru a asigura quorum-ul.

5.3.3 Scheduler (kube-scheduler)

Scheduler-ul este responsabil pentru alocarea Pod-urilor nou create pe nodurile disponibile ale clusterului. Decizia de planificare ia în considerare o serie de factori:

- resursele disponibile pe fiecare nod (CPU, memorie);
- cerințele de resurse ale Pod-ului (requests și limits);
- constrângerile de afinitate și anti-afinitate;
- politicile de toleranță la *taints*;
- prioritatea și clasa de calitate a serviciului (*QoS class*).

5.3.4 Controller Manager (kube-controller-manager)

Controller Manager este un proces care rulează un set de controllere responsabile pentru reconcilierea continuă a stării clusterului cu starea declarată. Fiecare controller monitorizează un tip specific de resursă și intervine atunci când starea reală diferă de cea dorită. Principalele controllere incluse sunt:

- **ReplicaSet Controller** — asigură că numărul de replici ale unui Pod corespunde valorii declarate;
- **Deployment Controller** — gestionează actualizările progresive ale aplicațiilor;
- **Node Controller** — monitorizează starea nodurilor și reacționează la defecțiunile acestora;
- **Endpoints Controller** — actualizează lista de endpoint-uri asociate fiecărui Service.

5.3.5 Cloud Controller Manager

Cloud Controller Manager este o componentă opțională care separă logica specifică furnizorului de cloud de nucleul Kubernetes. Aceasta gestionează resurse cloud precum load balancer-ele, discurile de stocare și adresele IP publice, prin API-urile specifice fiecărui furnizor (AWS, GCP, Azure etc.).

5.4 Componentele Worker Node

5.4.1 kubelet

kubelet este agentul Kubernetes care rulează pe fiecare nod din cluster. Acesta comunică permanent cu API Server și are responsabilitatea de a asigura că Pod-urile programate pe nodul respectiv sunt create, pornite și mențin starea de sănătate declarată. kubelet nu gestionează containere care nu au fost create prin Kubernetes.

5.4.2 kube-proxy

kube-proxy este un component de rețea care rulează pe fiecare nod și implementează regulile de rețea necesare pentru accesul la Service-uri. Acesta menține regulile *iptables* sau *IPVS* care permit routarea traficului de rețea către Pod-urile corecte, indiferent de nodul pe care acestea rulează.

5.4.3 Container Runtime

Container Runtime este componenta responsabilă cu executarea efectivă a containerelor pe nod. Kubernetes interacționează cu runtimes prin intermediul interfeței standardizate *Container Runtime Interface (CRI)*. Principalele implementări utilizate în prezent sunt:

- **containerd** — runtime recomandat, utilizat implicit de majoritatea distribuțiilor Kubernetes;
- **CRI-O** — runtime optimizat pentru Kubernetes, utilizat frecvent în OpenShift;
- **Docker Engine** — suport eliminat din Kubernetes începând cu versiunea 1.24.

5.5 Obiectele fundamentale Kubernetes

5.5.1 Pod

Pod-ul este unitatea atomică de planificare în Kubernetes. Un Pod poate conține unul sau mai multe containere care:

- partajează aceeași adresă IP și spațiul de porturi;
- au acces la același set de volume de stocare;
- sunt co-localizate pe același nod și co-planificate împreună.

În practică, un Pod conține de regulă un singur container principal, la care se pot adăuga containere auxiliare (*sidecar containers*) pentru funcționalități transversale (colectare de loguri, proxy, monitorizare).

5.5.2 ReplicaSet și Deployment

Un ReplicaSet asigură că un număr specificat de replici ale unui Pod rulează permanent în cluster. În cazul în care un Pod se oprește din cauza unei erori, ReplicaSet Controller creează automat un nou Pod pentru a menține numărul de replici declarat.

Deployment-ul este abstractizarea recomandată pentru gestionarea aplicațiilor *stateless* în Kubernetes. Acesta gestionează ReplicaSet-uri și adaugă funcționalități suplimentare:

- actualizări progresive (*rolling updates*) — înlocuirea graduală a Pod-urilor cu

noua versiune, fără downtime;

- posibilitate de rollback la versiunea anterioară;
- control asupra strategiei și vitezei de actualizare.

5.5.3 Service

Adresele IP ale Pod-urilor sunt efemere — se modifică la fiecare repornire. Obiectul Service oferă un punct de acces stabil pentru un set de Pod-uri, identificate printr-un *selector* de etichete. Tipurile principale de Service sunt:

- **ClusterIP** (implicit) — expune serviciul pe o adresă IP internă, accesibilă doar din interiorul clusterului;
- **NodePort** — expune serviciul pe un port static al fiecărui nod din cluster;
- **LoadBalancer** — provizionează un load balancer extern prin API-ul furnizorului de cloud;
- **ExternalName** — mapează serviciul la un nume DNS extern.

5.5.4 ConfigMap și Secret

ConfigMap permite stocarea configurațiilor non-confidențiale (URL-uri, parametri de conectare, setări de mediu) în afara imaginilor containerelor, facilitând portabilitatea aplicației între medii (dezvoltare, test, producție).

Secret este destinat stocării datelor sensibile (parole, token-uri de autentificare, chei API, certificate TLS). Valorile din Secret sunt codificate în Base64 și pot fi criptate la repaus prin mecanismele native Kubernetes sau prin soluții externe (HashiCorp Vault, AWS Secrets Manager).

5.5.5 Namespace

Namespace-urile permit împărțirea logică a unui cluster Kubernetes în medii virtuale izolate. Ele sunt utilizate pentru separarea resurselor între echipe, proiecte sau medii (dezvoltare, staging, producție) pe același cluster fizic, cu aplicarea de politici de acces și cote de resurse distincte pentru fiecare namespace.

5.5.6 Ingress

Ingress este un obiect Kubernetes care gestionează accesul HTTP și HTTPS din exteriorul clusterului la serviciile interne. Un Ingress Controller (NGINX, Traefik, HA-Proxy etc.) implementează regulile definite în obiectele Ingress, permițând rutarea bazată pe calea URL sau pe hostname, terminarea TLS și alte funcționalități de reverse proxy.

5.5.7 Persistent Volumes

Pentru aplicațiile care necesită stocare persistentă (baze de date, sisteme de fișiere), Kubernetes oferă abstracțiile **PersistentVolume (PV)** și **PersistentVolumeClaim (PVC)**. PV reprezintă resursa de stocare furnizată de administrator sau provizionată dinamic, iar PVC este cererea de stocare formulată de aplicație. Această separare de responsabilități permite portabilitatea aplicațiilor între diferiți furnizori de stocare.

6 Studiu de caz: Containerizarea unei aplicații bazate pe micro-servicii

6.1 Prezentarea aplicației

Pentru ilustrarea practică a conceptelor prezentate, este analizată o aplicație de administrare a unui club sportiv, dezvoltată pe o arhitectură de microservicii. Sistemul este alcătuit din componente independente care comunică prin API REST.

Microserviciile aplicației

Serviciu	Responsabilitate	Replici K8s
Auth Service	Autentificare și autorizare	3
User Service	Gestionarea profilurilor	2
Booking Service	Rezervări ore / terenuri	3
Notification Service	Trimitere notificări	2
Database	PostgreSQL, stocare persistentă	1

6.2 Containerizarea cu Docker

Fiecare microserviciu este împachetat într-o imagine Docker separată, cu propriul Dockerfile. Imaginile sunt stocate într-un registry privat și versionate prin taguri corespunzătoare versiunilor aplicației. Procesul de build este automatizat prin pipeline-uri CI/CD care validează, construiesc și publică imaginile la fiecare commit în ramura principală.

6.3 Implementarea în Kubernetes

Fiecare microserviciu este gestionat printr-un obiect Deployment și expus prin intermediul unui Service de tip ClusterIP. Comunicarea externă este gestionată de un Ingress Controller care rutează cererile HTTP/HTTPS către serviciile corespunzătoare.

Configurațiile specifice mediului (adrese de conectare la baza de date, parametri

de mediu) sunt stocate în obiecte ConfigMap, iar credențialele în obiecte Secret, ambele referențiate în specificațiile Deployment-urilor.

6.4 Scalare automată și toleranță la erori

Kubernetes Horizontal Pod Autoscaler (HPA) este configurat pentru Booking Service, asigurând scalarea automată a numărului de replici în funcție de utilizarea procesorului. Dacă utilizarea medie depășește 70%, noi replici sunt create automat, până la un număr maxim prestabilit.

În cazul defecțiunii unui nod, Controller Manager detectează situația și reprogramează Pod-urile afectate pe nodurile rămase disponibile. Procesul este transparent pentru utilizatorii finali, serviciile continuând să funcționeze prin intermediul celorlalte replici active.

7 Avantaje, provocări și tendințe

7.1 Avantajele utilizării containerelor și orchestrării

Adoptarea containerelor și a platformelor de orchestrare oferă beneficii semnificative:

- **Portabilitate** — aplicațiile rulează identic în orice mediu care dispune de runtime-ul containerelor;
- **Eficiența resurselor** — consumul de memorie și procesor este semnificativ redus față de mașinile virtuale;
- **Viteză de implementare** — ciclurile de livrare software se scurtează prin automatizarea build-ului și a deployment-ului;
- **Scalabilitate** — sistemul răspunde automat la variațiile de încărcare;
- **Disponibilitate ridicată** — mecanismele de self-healing minimizează impactul defecțiunilor.

7.2 Provocări și limitări

Utilizarea acestor tehnologii implică și provocări care trebuie gestionate:

- **Complexitate operațională** — administrarea unui cluster Kubernetes necesită competențe avansate în domenii precum rețelistică, stocare distribuită și securitate;
- **Securitate** — partajarea kernel-ului între containere introduce riscuri care ne-

cesită măsuri specifice (imagini verificate, scanare de vulnerabilități, politici de rețea);

- **Depanare** — identificarea cauzei problemelor în sisteme distribuite cu zeci de microservicii este mai complexă decât în aplicațiile monolitice;
- **Gestionarea stării** — aplicațiile *stateful* (baze de date, sisteme de mesagerie) sunt mai dificil de gestionat în Kubernetes față de cele *stateless*.

7.3 Tendințe și direcții viitoare

Ecosistemul containerelor și al orchestrării continuă să evolueze rapid. Principalele direcții identificate sunt:

- **Serverless containers** — abstractizarea completă a infrastructurii, în care furnizorul cloud gestionează automat alocarea resurselor (AWS Fargate, Azure Container Instances, Google Cloud Run);
- **eBPF și observabilitate avansată** — tehnologia eBPF permite implementarea de funcționalități de rețea și securitate direct în kernel, fără modificarea aplicațiilor;
- **Edge computing** — rularea containerelor pe dispozitive distribuite geografic, aproape de sursa datelor, pentru reducerea latenței;
- **Integrarea AI/ML** — utilizarea algoritmilor de învățare automată pentru optimizarea planificării resurselor și detectarea anomaliilor în clustere.

8 Concluzii

Lucrarea de față a analizat tehnologiile de containerizare și orchestrare a containerelor, evidențiind rolul esențial pe care acestea îl joacă în infrastructurile cloud-native moderne.

Containerele software, prin mecanismele de izolare oferite de kernel-ul Linux (*namespaces*, *cgroups*, *UnionFS*), reprezintă o alternativă eficientă față de mașinile virtuale tradiționale, oferind portabilitate ridicată, consum redus de resurse și viteze de deployment semnificativ mai bune.

Platforma Docker a standardizat procesul de containerizare, oferind un model simplu și reproductibil de construire, distribuire și rulare a aplicațiilor. Componentele sale — Dockerfile, imagini, containere, registry — formează un ecosistem coerent, integrat în fluxurile moderne de DevOps și CI/CD.

Kubernetes, ca platformă de orchestrare, adresează provocările administrării la scară a aplicațiilor containerizate. Arhitectura sa declarativă, mecanismele de self-healing, scalarea automată și suportul pentru actualizări fără downtime fac din Kubernetes componenta centrală a infrastructurilor cloud-native.

Studiul de caz a ilustrat modul concret în care aceste tehnologii se combină pentru a oferi o infrastructură flexibilă, scalabilă și rezistentă la erori pentru aplicații bazate pe microservicii.

În perspectivă, containerele și orchestrarea acestora vor continua să reprezinte fundamentul aplicațiilor distribuite, evoluând în direcția abstractizării complete a infrastructurii, a integrării cu tehnologiile de inteligență artificială și a extinderii către mediile de edge computing.

8 Bibliografie

-
- [1] Docker Inc., *Docker Documentation*, 2024. Disponibil la: <https://docs.docker.com>
 - [2] The Kubernetes Authors, *Kubernetes Documentation*, 2024. Disponibil la: <https://kubernetes.io/docs>
 - [3] N. Poulton, *Docker Deep Dive*. Independently Published, 2023.
 - [4] K. Hightower, B. Burns, J. Beda, *Kubernetes: Up and Running*, ediția a 3-a. O'Reilly Media, 2022.
 - [5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, "Borg, Omega and Kubernetes", *Communications of the ACM*, vol. 59, nr. 5, 2016.
 - [6] D. Bernstein, "Containers and Cloud: From LXC to Docker to Kubernetes", *IEEE Cloud Computing*, vol. 1, nr. 3, 2014.
 - [7] A. Verma et al., "Large-scale Cluster Management at Google with Borg", *Proceedings of EuroSys*, 2015.
 - [8] Cloud Native Computing Foundation, *CNCF Cloud Native Definition v1.0*, 2024. Disponibil la: <https://github.com/cncf/toc/blob/main/DEFINITION.md>
 - [9] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment", *Linux Journal*, nr. 239, 2014.
 - [10] J. Turnbull, *The Docker Book*. James Turnbull Publishing, 2023.